System One Glimmer

A calibrated, non-autoregressive tool-call guard for Meta's Muse Code

What we are building

A guard for autonomous coding agents. Before a tool call runs, it answers one narrow question — **will this command fail?** — and returns a *probability*, not a sentence. A threshold in application code turns that probability into **allow**, **ask** or **block**.

The claim under test is not that a big model is accurate. It is that **a raw next-token logit is not a decision**: you need a mechanism that forces the answer into a measurable quantity and a calibration step that makes the number mean what it appears to mean.

The mechanism — three commitments

1. **No generation.** One forward pass, then read two logits at the final position. We never sample a token, so latency is a single forward pass rather than autoregressive decode.
2. **No new head.** We mask the existing vocabulary — the weights are unchanged in shape, the adapter is 0.37% of parameters, and the approach ports to any checkpoint.
3. **Policy lives in code, not the model.** The model emits p(fail); allow/ask/block is a threshold the application owns. This is what makes coverage-at-a-risk-budget the honest metric.

How one decision is made

Prompt (from `src/noul.py`) → one forward pass → logits at the last real token → select **two** columns only → two-way softmax → **p(fail)** → thresholds → allow / ask / block.

Deterministic prechecks run *before* the model and **win**. If a string comparison proves a command destructive, a probability estimate does not get to overrule it. A 1B model should not be the thing standing between a user and `rm -rf /` when a regex gets it right every time.

Results so far

Llama-3.2-1B — the validated mechanism rung

Test split n=676, 199 failures (29.4%). Majority-class floor **70.6%**. Three seeds, three configurations.

config	seed 0	seed 1	seed 2	mean
zero-shot accuracy	64.9%	64.9%	64.9%	64.9%
+ LoRA	75.4%	75.6%	74.0%	75.0%
+ temperature	75.4%	75.6%	74.0%	75.0%

Accuracy by seed. Zero-shot is identical across seeds because scoring a frozen base model is deterministic.

METRIC	ZERO-SHOT	+ LoRA	+ TEMPERATURE
Brier	0.2800	0.203–0.208	0.186–0.188
ECE	0.2426	0.148–0.150	0.049 / 0.051 / 0.052
AURC	0.3621	0.207–0.210	0.207–0.210
cov@5%	0%	4–7%	4–7%

Calibration and selective prediction. A single fitted temperature cuts ECE by roughly 4.8x.

The claim worth making: accuracy lands +4.4 points above a strong floor with a 1.6-point spread across seeds, while one fitted temperature cuts ECE by ~0.19. **The calibration gain is an order of magnitude larger than the seed noise.** That is the result — not the accuracy.

Muse Glimmer 30B — the headline model, zero-shot

CONFIG	ACCURACY	BRIER	ECE	AURC	COV@5%
Glimmer 30B zero-shot	44.7%	0.3323	0.2504	0.5611	1%
always answer "ok"	70.6%	—	—	—	—
lift	-25.9 pts				

n=676, 4-bit NF4, single forward pass.

Read this plainly: the 30B model, asked directly, is worse than answering "ok" every time. Correlation with the label is +0.050 and AURC 0.5611 is worse than uninformative ranking. Scale does not solve this task. The mechanism plus a fitted temperature is what produces a usable probability.

Benchmarks

Both published benchmarks were converted to the same (prompt, label) shape as training and run through the same two-token read, so no benchmark gets a bespoke prompt.

BENCHMARK	N	TRIVIAL-BASELINE FLOOR	MAJORITY	LIFT	VERDICT
R-Judge	812	78.1%	59.6%	+18.5 pts	**FAIL**
Agent-SafetyBench	1,906	66.2%	66.2%	+0.0 pts	OK

A benchmark a constant or length-only predictor can match cannot support a claim about model quality.

R-Judge fails the trivial-baseline test *even after* being reduced from whole interactions to 812 individual agent actions. Its label remains confounded with interaction length at step granularity. It is therefore **eval-only**, with the floor printed beside every number, and never a training signal. This is the same falsification that killed it as training data, measured a second time at finer granularity.

Bugs found by testing, not inspection

Every item below was caught by a test or a measurement. Each now has a guard that fails in under a second instead of after a 12-minute cluster job.

DEFECT	HOW IT WAS HIDING	NOW PINNED BY
Label convention inverted	Prompt said "yes = fail" but the target for a failing example was the token id of "no". No exception, plausible loss curve. Three training rounds were built on it.	`tests/test_noul_contract.py` — 14 tests
Metrics overstated coverage	The trainer carried its own `cov_at` that checked only the last prefix of the risk-coverage curve. Every published cov@5% came from it.	`test_no_duplicate_metrics.py`
TypeError after 22 min of GPU	`coverage_at` returns (fraction, n); the reporter formatted it as a scalar.	`test_summarize_returns_only_scalars`
Loss read outside its scope	Read after the accumulation loop closed; surfaced 12 min into a cluster job.	`test_no_variable_is_read_outside_its_scope`
exp() overflow in the guard	`math.exp(-900)` raised — the guard would have crashed exactly when the model was most confident.	`test_llamacpp_client.py`
Data leak	606 identical prompts appeared in both train and test, inflating the score.	leak gate inside `make check`

Six defects, none visible by reading the diff.

Infrastructure problems worth recording

Three cost hours and none were code bugs:

- **Amarel `gpuk*` nodes mount a different filesystem.** `/scratch` there resolves to `/scache` (257T) and is *invisible from the login node*. Jobs reported COMPLETED while writing output into a void. Every allocation now excludes them.
- **`sbatch --wrap` through ssh collapses quoting.** `scontrol` showed `Command=(null)`, so the job ran nothing at all. Batch scripts only.
- **Compute nodes have no git and no cmake.** Source comes from a tarball; cmake is installed into a venv via uv. Loading `hpc_sdk` also puts `nvc++` ahead of gcc, which CMake then picks as the host compiler — it rejects gcc-only warning flags and the build dies deep inside ggml, nowhere near CUDA code.

Status against the goal

DONE-CRITERION	STATE	EVIDENCE
All tests in make check pass	**done**	131 tests, ALL CHECKS PASSED
Noul-style decision model, single-token masking, no generation	**done**	proven on both tokenizers; Ġyes=19690, Ġno=916
1B: 3 configs, 3 seeds	**done**	75.0% mean vs 70.6% floor; ECE 0.2426 → 0.049
Glimmer 30B headline model	**partial**	mechanism proven; zero-shot 44.7%; fine-tune running
R-Judge + Agent-SafetyBench, 3 configs each	**partial**	harness built and floor-tested; adapter scores pending
Muse Code integration with p50/p95	**partial**	guard policy + HTTP client tested; llama-server building
223k step-level dataset	**will not be met**	6,764 unique commands from 19,210 raw pairs
Public repo + licences + README	**blocked**	ready locally, 11 commits; awaiting your approval
Muse Code intercepting real tool calls	**not started**	needs the server and a GGUF

Deliberately not rounded up: four criteria are partial and one is unreachable.

> **The one criterion that will not be met.** The goal asks for a 223k step-level dataset. After deduplication we have 6,764 unique commands from 19,210 raw pairs — 12,446 collapsed as exact duplicates and 1,575 are genuine label conflicts where the same command both worked and failed. The 223k figure is not reachable from this source, and padding it would be fabrication. We will publish the real number with the raw count as provenance.

What remains

1. Finish the Glimmer QLoRA run and report the fine-tuned 30B numbers against the 1B rung. The open question is whether 30B fine-tuned beats 75.0% or whether 1B was already near the task ceiling.
2. Score both benchmarks for all three configurations using the saved adapters.
3. Quantise the merged adapter and serve it from llama-server; then wire Muse Code with `--provider local --base-url` and measure added latency p50/p95.
4. Produce the demo: baseline cost, guarded run, offline run.
5. Publish the repository — blocked on your approval.